Eric Romero
CSC 345
2/2/2026

Homework 2

Idea: Given n sets, each with a size of at most k, for every pair of sets that share at least one element, compute sim(A, B) = |A ∪ B| / |A ∩ B|.

Return only the non zero ones.

General Case:

Algorithm Description:

Compare every pair of sets. For each pair, count how many elements they share by scanning the smaller set and checking its existence in the other set. That count will be the intersection size. Compute the union size using the inclusion-exclusion principle. If the intersection count is 0, skip the pair. Otherwise, compute the fraction and store under the pair (i, j).

Pseudocode:

```
similarities_general(setList):
    n ← size(setList)
    result ← empty map (i → map(j →double))

    sizes[i] ← |setList[i]| for all i

    for i = 0 to n-1:
        for j = i+1 to n-1:
            A ← setList[i]
            B ← setList[j]

            if |A| > |B|:
                swap A, B

            inter ← 0
            for each x in A:
                if x in B:
                    inter ← inter + 1

                if inter > 0:
                    union ← sizes[i] + sizes[j] - inter
                    sim ← inter / union
                    result[i][j] ← sim
    return result
```

Correctness Proof:

1) For any pair (i, j), the algorithm solves for the right intersection size

First, choose one of the given sets S_i or S_j. The algorithm will iterate through every element x in the smaller set of the two. For each x, it checks if it exists in the other set.

Eric Romero
CSC 345
2/2/2026

If $x \in S_i \cap S_j$, the existence test will proceed and the intersection size variable is incremented.

If $x \notin S_i \cap S_j$, do not increment.

Assuming these sets do not contain duplicates, each shared element is counted only once, and no element that is not shared is counted.

Therefore, the loop finished with: inter = $|S_i \cap S_j|$

2) Given the intersection, the algorithm solves for the right union size

Through a standard identity for sets:

$|S_i \cup S_j| = |S_i| + |S_j| - / |S_i \cap S_j|$

The algorithm solves the union with 'sizes[i] + sizes[j] - inter.

Because 'sizes[i]' is equal to $|S_i|$, 'sizes[j]' is equal to $|S_j|$, and by the first claim, 'inter = $|S_i \cap S_j|$', then the solved union is equal to $|S_i \cup S_j|$.

3) The similarity that is written to the output is right, and the output sparsity is correct.

If inter = 0, then the similarity is '0 / $|S_i \cup S_j|$ = 0'. Skipping it will match the requirement to only store nonzero similarities.

If inter > 0, the algorithm stores:

Sim = inter / union = $|S_i \cup S_j|$ / $|S_i \cap S_j|$.

This exactly defines similarity. It only considers i < j, so never includes duplicates or self-pairs.

Therefore, every stored similarity is correct and every ignored similarity is ignored.

Complexity Analysis:

Time Complexity:

Let n be the number of sets,with each set size less than or equal to k.

This algorithm will examine all pairs results in (n / 2) = O(n^2)

For each pair, we iterate the smaller set. In the worse case (both sets have size k), the intersection counting costs O(k).

Result: $O(n^2k)$

Space Complexity:

This algorithm stores an array of length n (O(n)) and an output map of nonzero similarities.

Eric Romero
CSC 345
2/2/2026

In this case, many pairs can overlap, so up to $O(n^2)$ pairs are stored.

Without output: $O(n)$
With output: $O(n^2)$

Eric Romero
CSC 345
2/2/2026

Sparse Similarity Case:

Algorithm Description:

First, scan the input sets and build a map from each element to all sets it appears in. This is an inverted index. This should show that for each element, these are the sets that are guaranteed to overlap because of that element.

Next, for each element x, take its posting list (the indices of sets containing x) and get all pairs of (i, j) from that list where i < j. For each of these pairs, increment a counter that tracks how many shared elements the two sets have. Because each element contributes exactly one to the intersection of whichever pair that contains it, the counters become true intersection sizes.

Lastly, Walk through every pair (i, j) that has a positive count and solve for similarity: sim(i, j) = inter(i, j) / ($|S_i|$ + $|S_j|$ - inter(i, j)

Correctness Proof:

Lemma 1: Every increment corresponds to a real shared element.

An increment to interCount[i][j] happens only when we process some element x whose posting list contains both i and j. That means x is an element of S_i and x is an element of S_j, so x is genuinely in the intersection S_i (intersection) S_j So we never “invent” intersection elements that don’t exist.

Lemma 2: Every shared element contributes exactly one increment to the right pair.

Take any pair of sets (S_i , S_j) and any element x that is an element of S_i (intersection) S_j. When we build the inverted index, i and j are both placed into index[x]. Later, when we process index[x], the nested loop enumerates the pair (i, j) exactly once (using i < j), and increments interCount[i][j] by 1. Since sets contain no duplicate elements, x causes exactly one increment.

Lemma 3: After processing all elements, interCount[i][j] = $|S_i \cap S_j|$ for every overlapping pair.

By Lemma 1, interCount[i][j] counts only real shared elements. By Lemma 2, every shared element contributes exactly one to the count. Therefore the total count equals the number of shared elements: interCount[i][j] = $|S_i \cap S_j|$

Lemma 4: The similarity computed for each stored pair is correct.

For each stored pair, we compute:

- inter = $|S_i \cap S_i|$ (Lemma 3)
- union = $|S_i|$ + $|S_j|$ − inter = $|S_i \cup S_j|$

So: sim = inter / union = $|S_i \cap S_j|$ / $|S_i \cup S_j|$, which matches the definition. We also store only pairs with inter > 0, so the output is correctly sparse.

Therefore the algorithm returns exactly the nonzero similarities, and each value is correct.

Complexity Analysis:

Time Complexity:

1) Build the “element → sets” map (inverted index)

Walk through every element inside every set exactly once.

- If there are n sets
- and each has at most k elements

then you touch at most n \cdot k element occurrences.

Time for this phase: O(nk)

2) Count intersections by pairing sets that share an element

Now for each element x, suppose it appears in f_x sets.

Then you get all pairs among those sets: f_x / 2.

Do this for every element.

Time for this phase: O(Sum_x for (f_x / 2)

Space Complexity: